

ROS



POLITECNICO
MILANO 1863



ABOUT ME



Mentasti Simone, researcher at AIRLab

Contacts:

simone.mentasti@polimi.it

Research field:

Autonomous vehicles

Autonomous driving

Sensor fusion

Deep learning

...





Slide and example link:

<https://goo.gl/GonArW>

Schedule



L1	Middleware for robotics and ROS Installation “Party”	L6	rospy, rosbag, message filter
L2	Ros workspace, publisher/subscriber, tools	L7	ROS on multiple machines, time synchronization, Actionlib, latched pub, async spinner
L3	Publisher, subscriber, launch file , custom messages	L8	Robot Navigation, Stage, Gmapping
L4	Services, parameters, timers	L9	Robot Navigation (Part II), Robot Localization, mapviz
L5	TF, Rviz, node architecture, first project	L10	ROS2, second project

ROBOTIC MIDDLEWARES

ROBOTICS



POLITECNICO
MILANO 1863

MIDDLEWARE ORIGINS



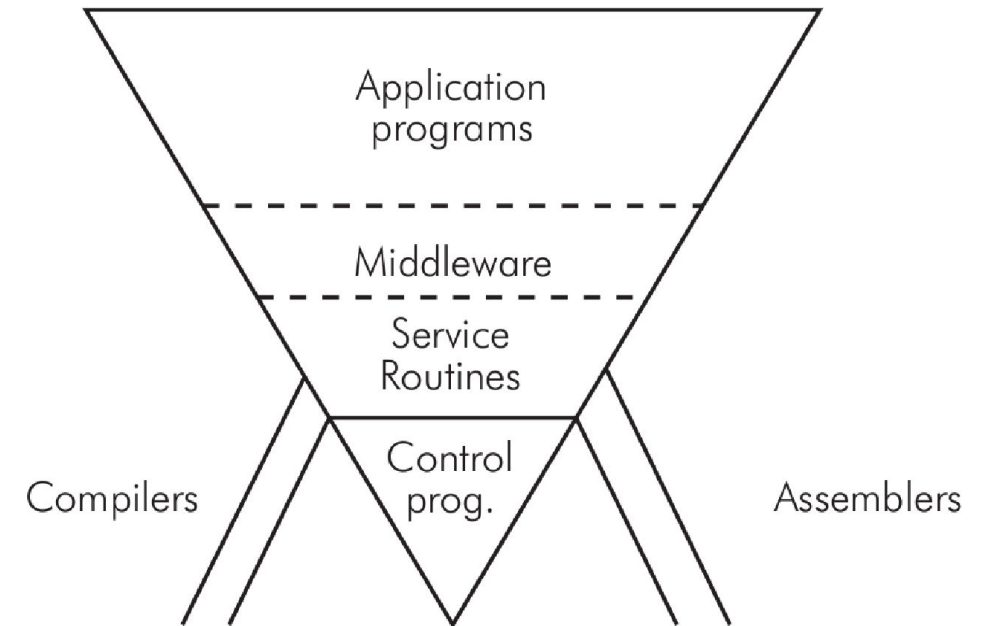
The origins

1968 introduced by d'Agapeyeff

80's wrapper between legacy systems and new applications

Nowadays: widespread in different domain fields (including Robotics)

Some (non robotics) examples: Android, SOAP, Web Services, ...



MIDDLEWARE ORIGINS



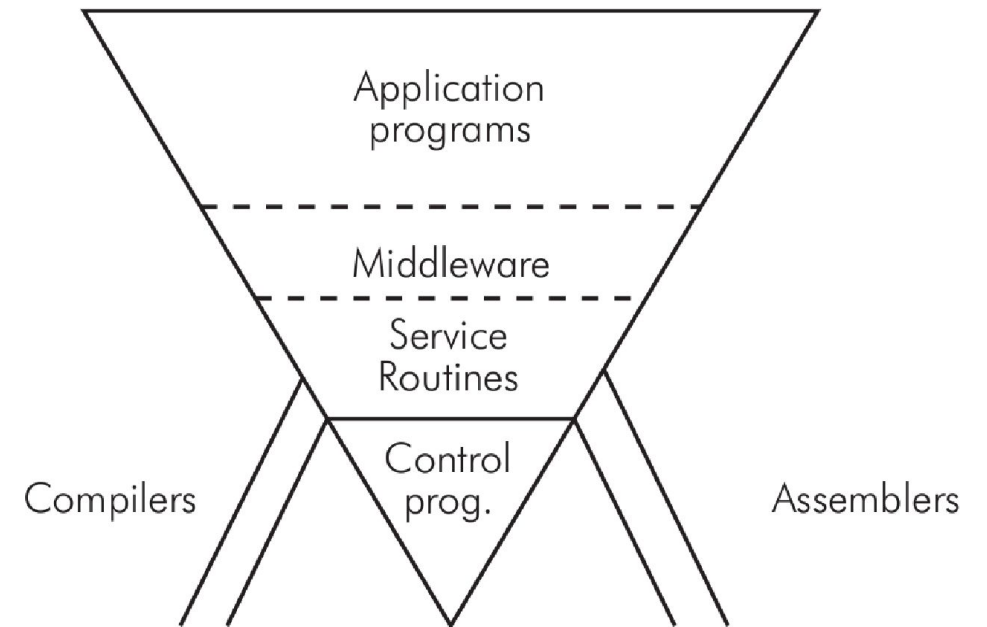
The Middleware idea

Well-known in software engineering

It provides a computational layer

A bridge between the application
and the low-level details

It is not a set of API and library



MIDDLEWARE ORIGINS

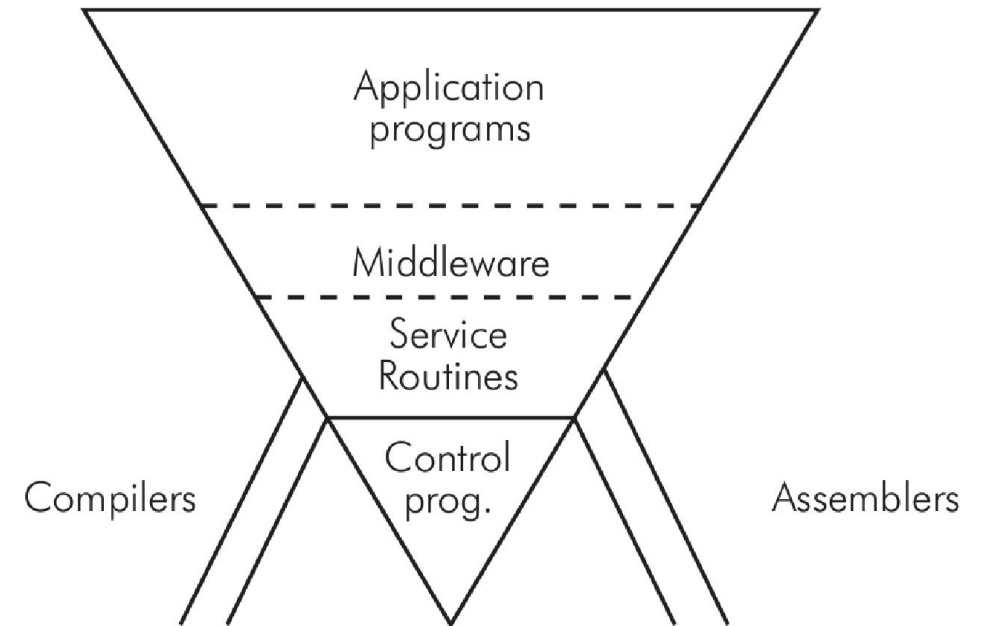


Issues in developing real robots

Cooperation between hardware and software

Architectural differences in robotics systems

Software reusability and modularity



MIDDLEWARES MAIN FEATURES



Portability: provides a common programming model regardless the programming language and the system architecture.

Reliability: middleware are tested independently. They permit to develop robot controllers without considering the low level details and using robust libraries.

Manage the complexity: low-level aspects are handled by libraries and drivers inside the middleware. It (should) reduce(s) the programming error and decrease the development time.

ROBOT MIDDLEWARES: A LIST



Several middleware have been developed in recent years:

OROCOS	[Europe]
ORCA	[Europe]
YARP	[Europe / Italy]
BRICS	[Europe]
OpenRTM	[Japan]
OpenRave	[US]
ROS	[US]

...

Let's see their common features and main differences

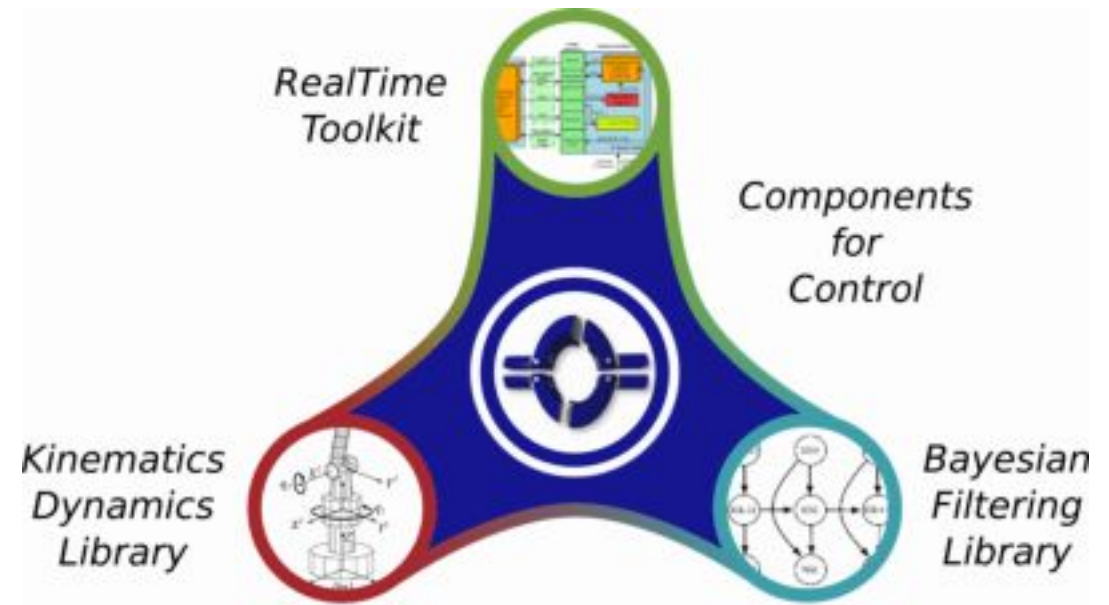
OROCOS: OPEN ROBOT CONTROL SOFTWARE



The project started in December 2000 from an initiative of the mailing list EURON then it become an European project with 3 partners: K.U. Leuven (Belgium), LAAS Toulouse (France), KTH Stockholm (Sweden)

OROCOS requirements:

- Open source license
- Modularity and flexibility
- Not related to robot industries
- Working with any kind of device
- Software components for kinematics, dynamics, planning, sensors, controller
- Not related to a unique programming language



OROCOS STRUCTURE

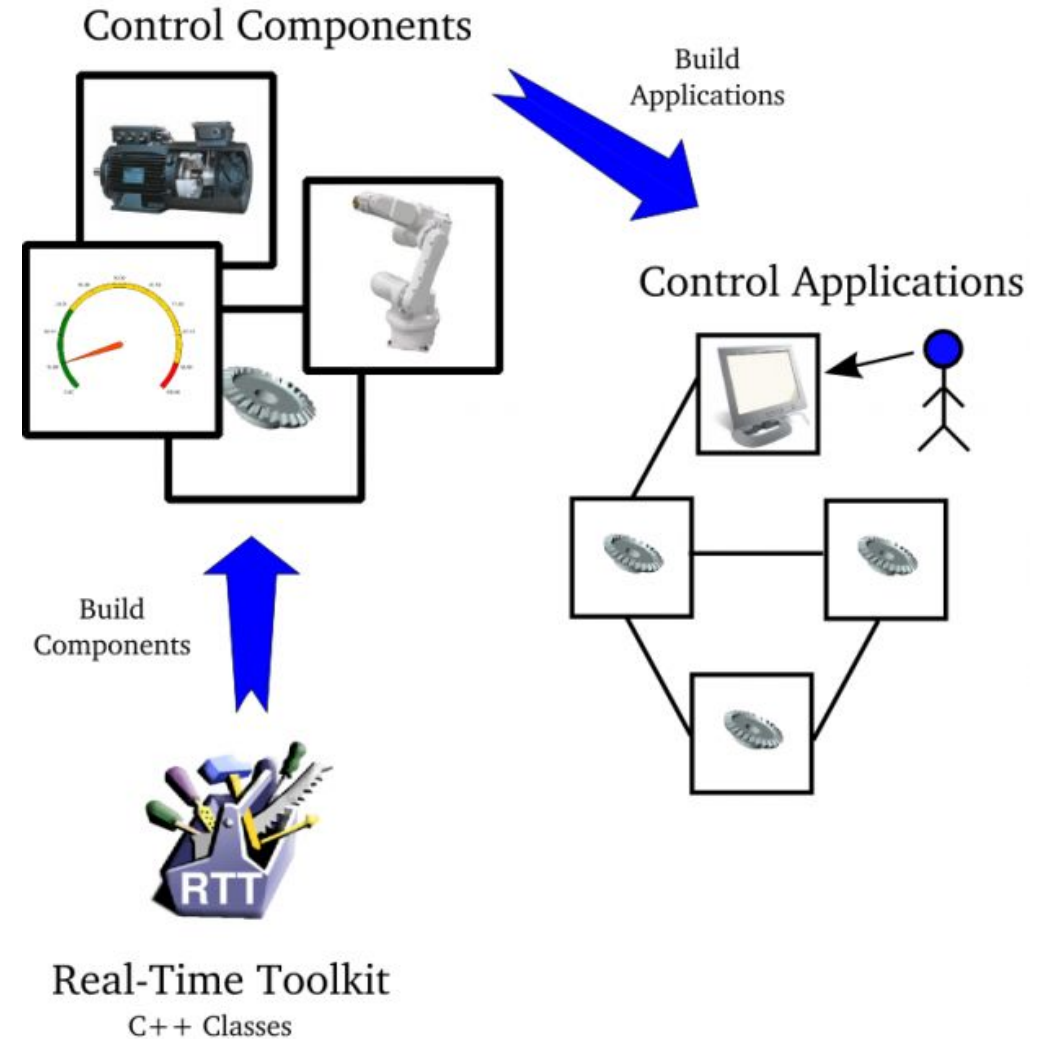


Real-Time Toolkit (RTT)

infrastructure and functionalities
for real-time robot systems
component-based applications

Component Library (OCL)

provides ready-to-use components,
e.g., device drivers, debugging tools,
path planners, task planners



OROCOS STRUCTURE

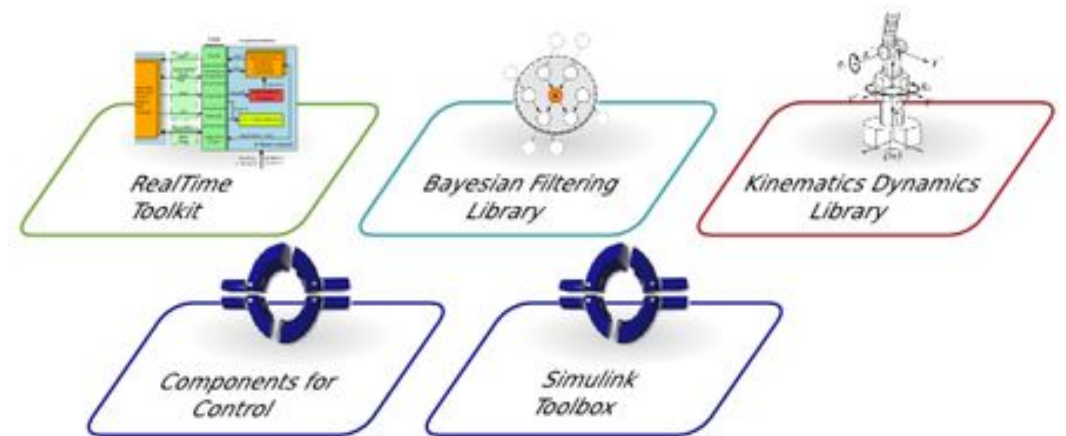


Bayesian Filtering Library (BFL)

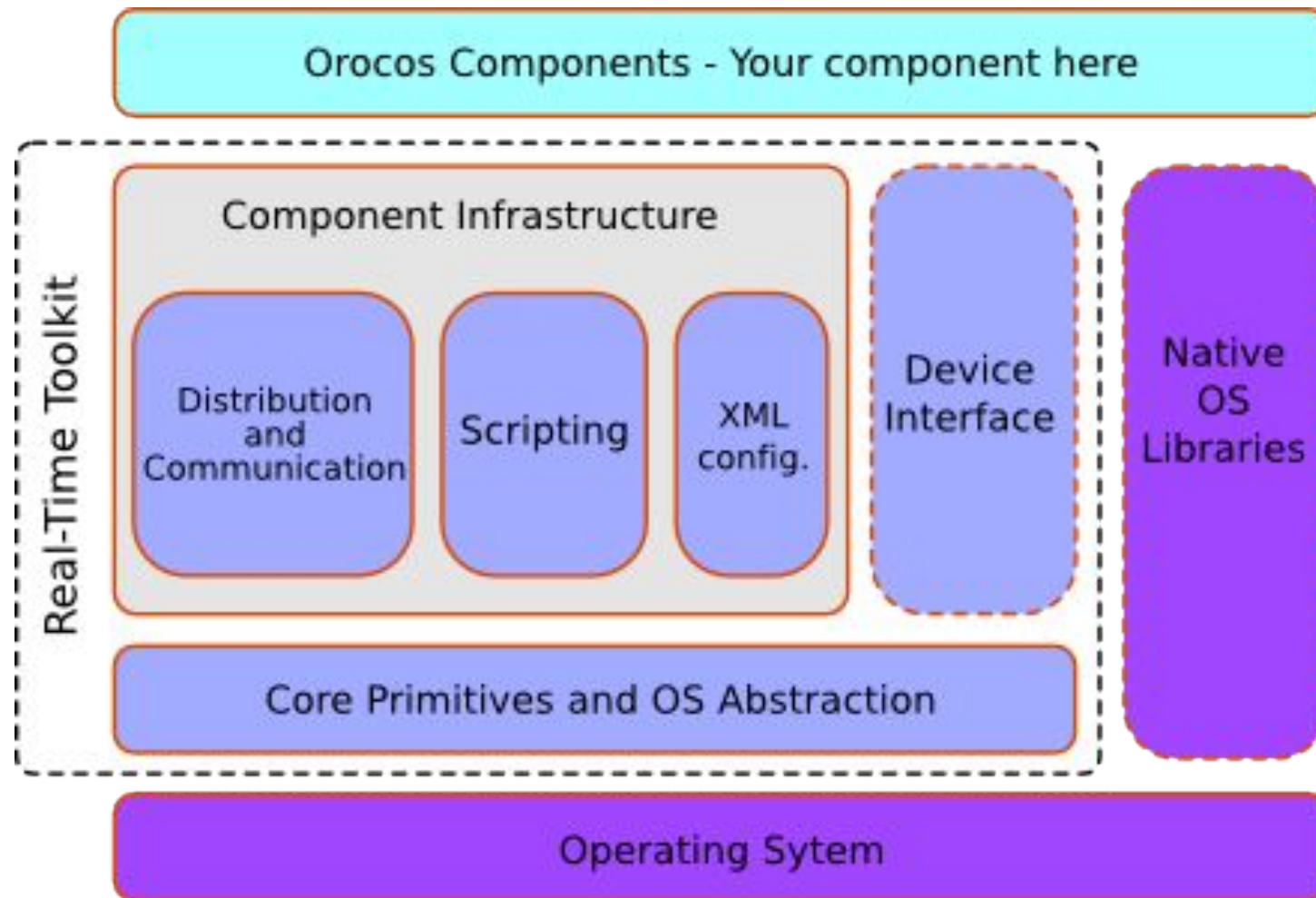
application independent framework,
e.g., (Extended) Kalman Filter,
Particle Filter

Kinematics & Dynamics Library (KDL)

real-time kinematics & dynamics
computations



OROCOS RTT FRAMEWORK



ORCA: COMPONENTS FOR ROBOTICS

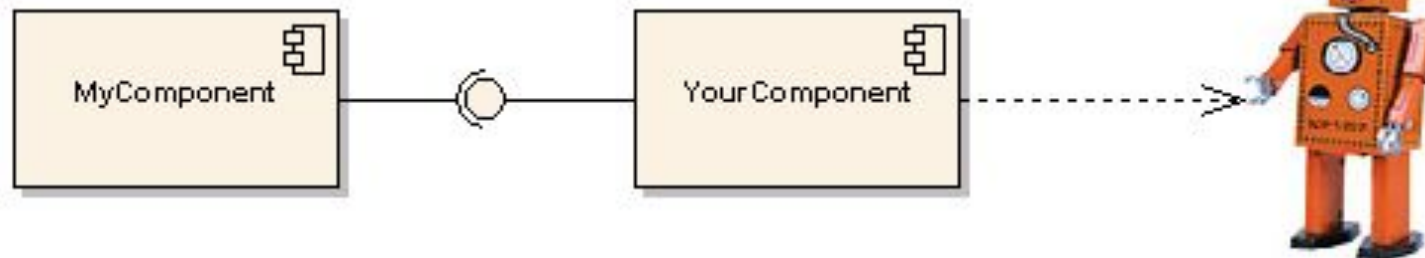
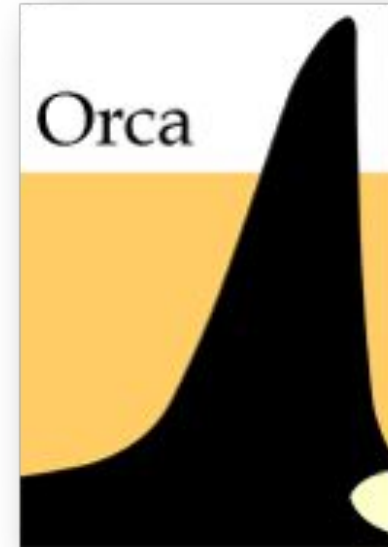


The aim of the project is to focus on software reuse for scientific and industrial applications

Key properties:

- **Enable software reuse** defining commonly-use interfaces
- **Simplify software reuse** providing high-level libraries
- **Encourage software reuse** updated software repositories

ORCA defines itself as “unconstrained component-based system”



ORCA AND ICE



The main difference between OROCOS and ORCA is the communication toolkit; OROCOS uses CORBA while ORCA uses ICE

ICE is a modern framework developed by ZeroC

ICE is an open-source commercial communication system

ICE provides two core services

IceGrid registry (Naming service): which provides the logic mapping between different components

IceStorm service (Event service): which constitute the publisher and subscriber architecture



“A component can find the other components through the IceGrid registry and can communicate with them through the IceStorm service.”

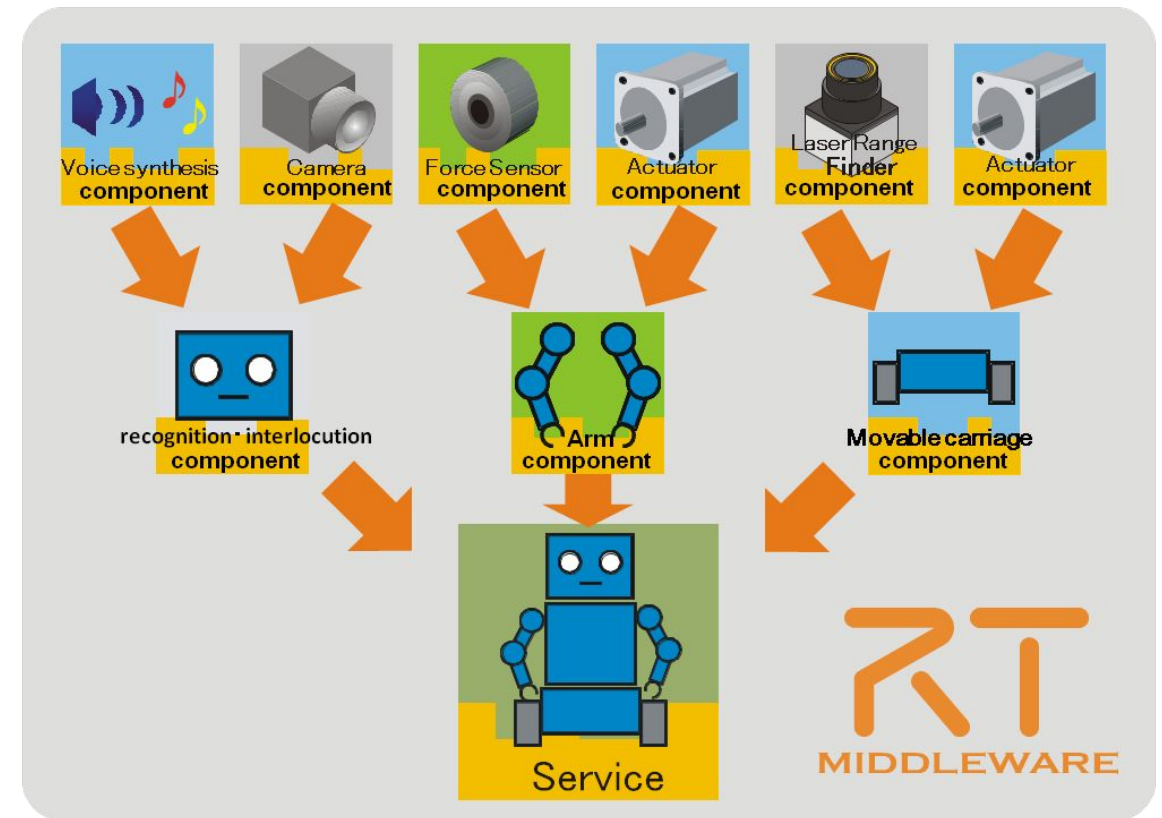
RT MIDDLEWARE



RT-Middleware (RTM) is a common platform standard to construct the robot system by combining the software modules of the robot functional elements (RTC):

- Camera component
- Stereovision component
- Face recognition component
- Microphone component
- Speech recognition component
- Conversational component
- Head and arm component
- Speech synthesis component

OpenRTM-aist (Advanced Industrial Science & Technology) is based on the CORBA technology to implement RTC extended specification

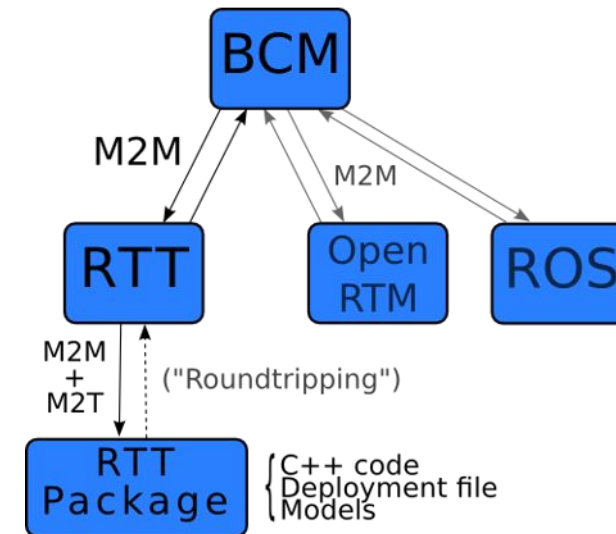


BRICS: BEST PRACTICES IN ROBOTICS



Aimed at find out the "best practices" in the developing of the robotic systems:

- Investigate the weakness of robotic projects
- Investigates the integration between hardware & software
- Promote model driven engineering in robot development
- Design an Integrated Development Environment for robotic projects (BRIDE)
- Define showcases for the evaluation of project robustness with respect to BRICS principles.

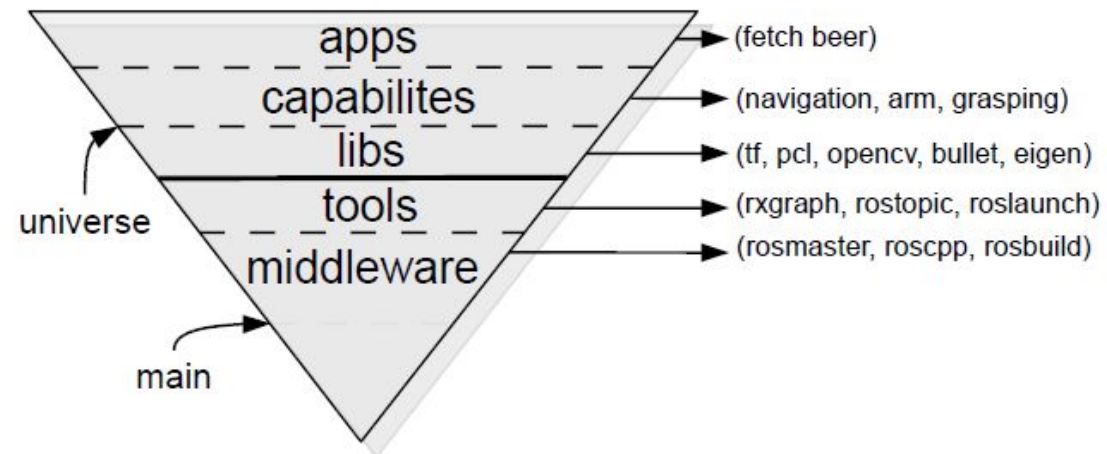


"The prime objective of BRICS is to structure and formalize the robot development process itself and to provide tools, models, and functional libraries, which help accelerating this process significantly."

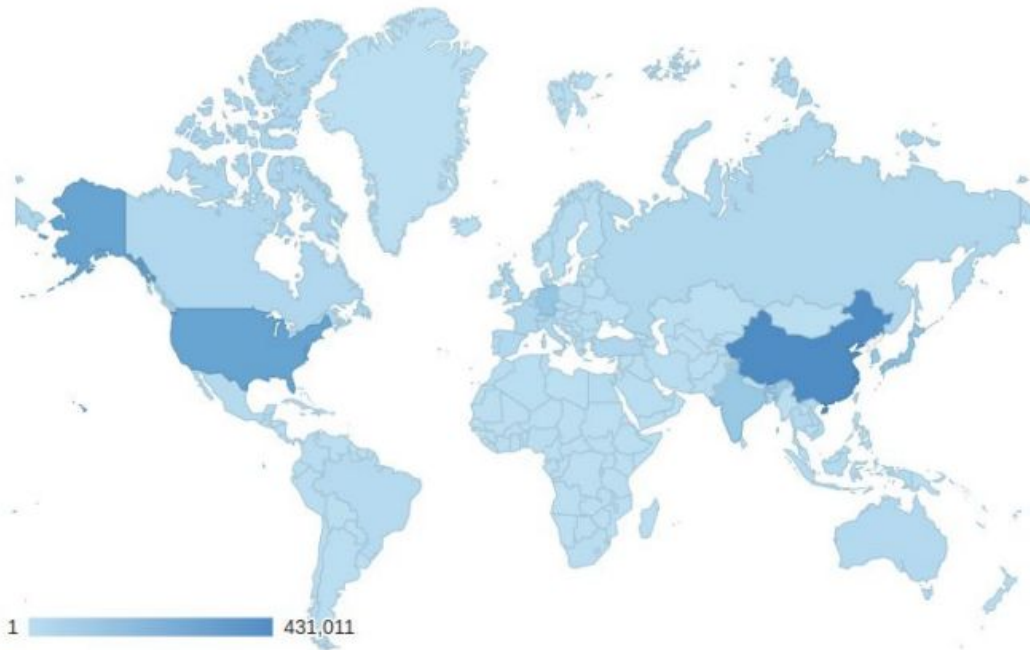
ROS: ROBOT OPERATING SYSTEM



Presented in 2009 by Willow Garage, is a meta-operating system for robotics with a rich ecosystem of tools and programs



WHY ROS?



wiki.ros.org

Traffic volume by country

Country	Acquisition	
	Users	
	291,342	% of Total: 100.00% (291,342)
1. United States	51,149	(16.88%)
2. China	47,317	(15.82%)
3. Germany	23,336	(7.90%)
4. Japan	20,908	(6.99%)
5. South Korea	17,076	(5.64%)
6. India	15,847	(5.23%)
7. United Kingdom	7,067	(2.33%)
8. France	6,653	(2.20%)
9. Taiwan	6,593	(2.18%)
10. Hong Kong	6,483	(2.14%)
11. Canada	6,189	(2.04%)
12. Italy	5,932	(1.96%)
13. Singapore	5,651	(1.88%)
14. Spain	5,009	(1.65%)
15. Russia	4,964	(1.64%)
16. Turkey	4,683	(1.55%)
17. Netherlands	4,580	(1.51%)
18. Australia	4,239	(1.40%)
19. Brazil	3,860	(1.27%)
20. Thailand	3,620	(1.19%)

ROS has grown to include a large community of users worldwide

The community of developer is one of the most important characteristics of ROS

<https://discourse.ros.org/uploads/short-url/lHSkkHpOngOJ2qvrNm1Nr5CGEU4.pdf>

ROS

A LOT OF RESOURCES



ROS Wiki

Archive for the existing ROS component
Installation and configuration guides
Information about the middleware itself
Lots of tutorials



ROS Q&A

For specific problems
Thousand of already answered questions
Active community
Like *Stack Overflow* for ROS

SOME NUMBERS



ROS wiki:

pages: 17058

edits: 14,7/day

views: 44794/day

ROS Q&A:

total Q: 30243

total A: 21697

avg Q: 17,2/day

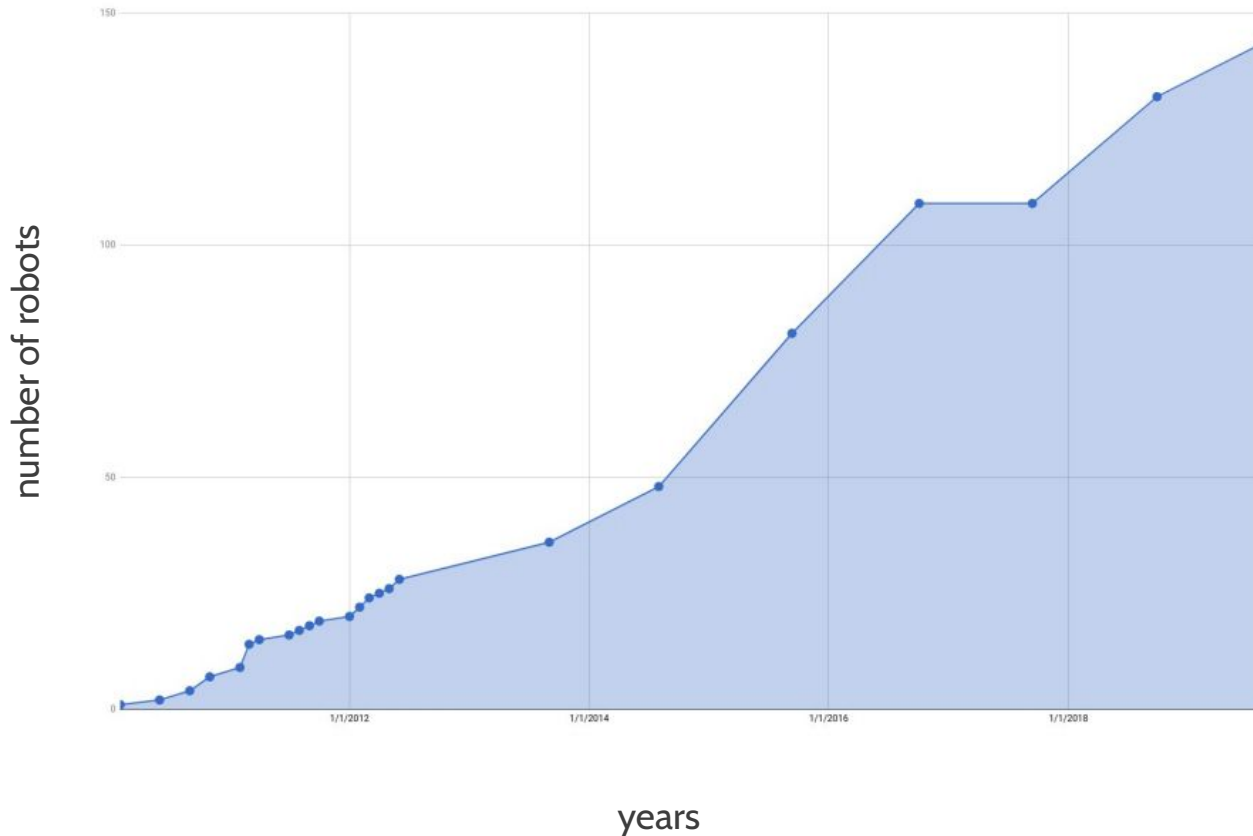
ROS deb:

total DL: 8441279

unique DL: 7582

unique IP: 113345

ROBOT AND RESEARCH



Total number of papers citing
*ROS: an open-source Robot
Operating System*
(Quigley et al., 2009)
10842(scholar)

ROS Industrial





Open Source Robotics Foundation (OSRF)

Open Source Robotics Corporation (OSRC)

ROS INTRODUCTION

ROBOTICS



POLITECNICO
MILANO 1863

Statistics



Distro	10/2021	10/2022	YoY Change
ROS 1 Kinetic	4.74%	2.62%	-2.62%
ROS 1 Melodic	36.64%	17.19%	-19.45%
ROS 1 Noetic	24.35%	<u>34.80%</u>	+10.45%
ROS 2 Humble	NA	10.04%	NA
ROS 2 Galactic	7.63%	8.12%	+0.49%
ROS 2 Foxy	14.04%	13.59%	-0.45%
ROS 2 Rolling	1.84%	3.38%	+1.54%
All other distros	2.24%	1.43%	-0.81%

INSTALLATION



This instructions are for:
Ubuntu 20.04 (suggested)



INSTALLATION (Linux users)

- Ubuntu 20.04 -> ROS Noetic

Different version of Ubuntu?

- Ubuntu older than 20.04 -> Specific ROS version (only issue with python)
- Ubuntu newer than 20.04 -> Distrobox

Different Linux Distro?

- Distrobox



INSTALLATION (Windows users)

- Dual boot is still the best option
- you can put linux on an external ssd, so nothing really change on your laptop
- Install Ubuntu 18.04 on an external disk (caution: experimental) [guide](#) (but use Ubuntu 18.04)
- Virtual machine (decent at the beginning, then maybe problematic)



INSTALLATION (MacOs users)

Older models (no M1 processor):

- should be able to dual boot

New models (M1 processor):

- Distrobox might work (really experimental)
- Fully docker based system (sometimes you might have to build again the image)
- Virtual machine (the easy solution, not the best solution)
- ROS might work directly on MacOS (not really)

INSTALLATION



If you have Ubuntu/Debian:

- <http://wiki.ros.org/noetic/Installation/Ubuntu>



INSTALLATION (distrobox)

If you have other linux distro/Ubuntu version:

Install Docker: <https://docs.docker.com/engine/install/>

Install Distrobox: <https://github.com/89luca89/distrobox#installation>

Then:

- `distrobox create robotics -i smentasti/robotics:latest`
- `distrobox enter robotics`

You can edit your `.bashrc` file to load variables only inside distrobox

You can configure a different terminal program to automatically load distrobox

Example terminator config provided (edit `~/.config/terminator/config`)



INSTALLATION (a note on docker)

To test the docker installation you can run the example container:

- `docker run hello-world`

If you get permission denied usually is due to groups settings:

- `sudo groupadd docker`
- `sudo usermod -aG docker $USER`

Reboot



.bashrc configuration

```
if [ "$HOSTNAME" = simone-xmg ]; then
    source /opt/ros/noetic/setup.bash
    source /home/simone/catkin_ws/devel/setup.bash --extend
    source /home/simone/catkin_devel/devel/setup.bash --extend
fi
```

```
if [ "$HOSTNAME" = deepfield.simone-xmg ]; then
    source /opt/ros/noetic/setup.bash
    source /repos/orb_slam/devel/setup.bash --extend
fi
```

```
if [ "$HOSTNAME" = robotics.simone-xmg ]; then
    source /opt/ros/noetic/setup.bash
    source /home/simone/robotics/devel/setup.bash
fi
```



.bashrc configuration

```
if [ "$HOSTNAME" = simone-xmg ]; then
    source /opt/ros/noetic/setup.bash
    source /home/simone/catkin_ws/devel/setup.bash --extend
    source /home/simone/catkin_devel/devel/setup.bash --extend
fi
```

Your system

```
if [ "$HOSTNAME" = deepfield.simone-xmg ]; then
    source /opt/ros/noetic/setup.bash
    source /repos/orb_slam/devel/setup.bash --extend
fi
```

deepfield distrobox

```
if [ "$HOSTNAME" = robotics.simone-xmg ]; then
    source /opt/ros/noetic/setup.bash
    source /home/simone/robotics/devel/setup.bash
fi
```

robotics distrobox



INSTALLATION (Docker)

First install docker:

- <https://docs.docker.com/engine/install/>
- Build an image using a Dockerfile (provided, it should have most of the required packages, run the command in the Dockerfile folder):
 - `docker build -t robotics:stable .`
- **install docker compose:**
 - `sudo apt install docker-compose`
- **run your image with docker compose:**
 - `docker-compose -f robotics.yaml up`
- **connect to the running image:**
 - `docker exec -it --user simone robotics_app /bin/bash`



INSTALLATION (Docker, how to get a gui?)

Docker usually works without gui, you do most of your work from terminal

In robotics you might need a gui (data visualization, debugging, etc.):

<http://wiki.ros.org/docker/Tutorials/GUI>

you have a vnc.sh script to start the vnc server, you can run it and then connect to

<http://localhost:8080/vnc.html>

For visualization Foxglove is multi-platform, and connect to running ROS on the same network/pc (more on this in the next lecture):

<https://foxglove.dev/>



Why all this talking on Docker? (Project)

This year the project will be “automatically” evaluated

You will get a Docker image, and a demo-evaluation script and the project will have to compile and run inside the Docker image i provide

So this year no python

Standard evaluation metrics will be used to evaluate the project



Challenges

Alternative to doing the projects

Require more time than doing the project

Experiment on real robots

Will have soft deadlines (but we have deadlines)

Competitions are after the end of the course, we still evaluate when the course finish (not on the challenge results)

ROS2 based

If interested, have question, etc. write a mail to us:

- Subject "Challenge Robotics 2023 - Leonardo"/"Challenge Robotics 2023 - FRE"
- Attach study plan
- write a few lines where you explain why you are interested in the challenge



UGV-UAV cooperation (Leonardo drone contest)

Organized by Leonardo

4th edition, 1st edition with UGV

Tasks:

- exploration
- mapping
- detection
- tracking
- cooperation

Urban-like environment

Competition in October

mail to: simone.mentasti@polimi.it





Agriculture robot (Field Robotics Event)

Agriculture competition

2nd year for our lab

newly built custom robot

tasks:

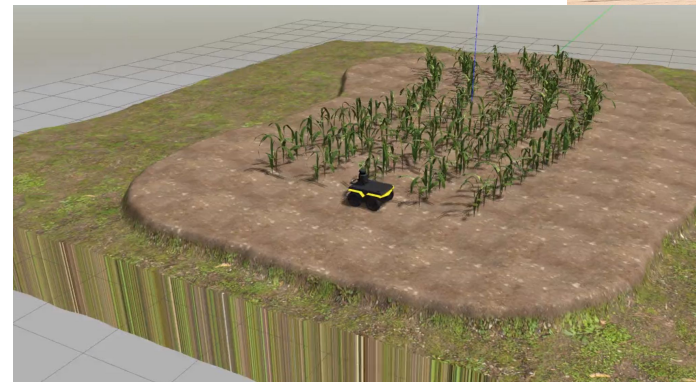
- autonomous navigation
- obstacle detection
- weed detection
- autonomous spraying

Competition in June (maybe test in May)

mail to: paolo.cudrano@polimi.it

simone.mentasti@polimi.it

International
FRE
Field Robot Event



ROS STRUCTURE

ROBOTICS



POLITECNICO
MILANO 1863

ROS: ROBOT OPERATING SYSTEM



ROS main features:

Distributed framework

Reuse code

Language independent

Easy testing on Real Robot & Simulation

Scaling

ROS Components

File system tools

Building tools

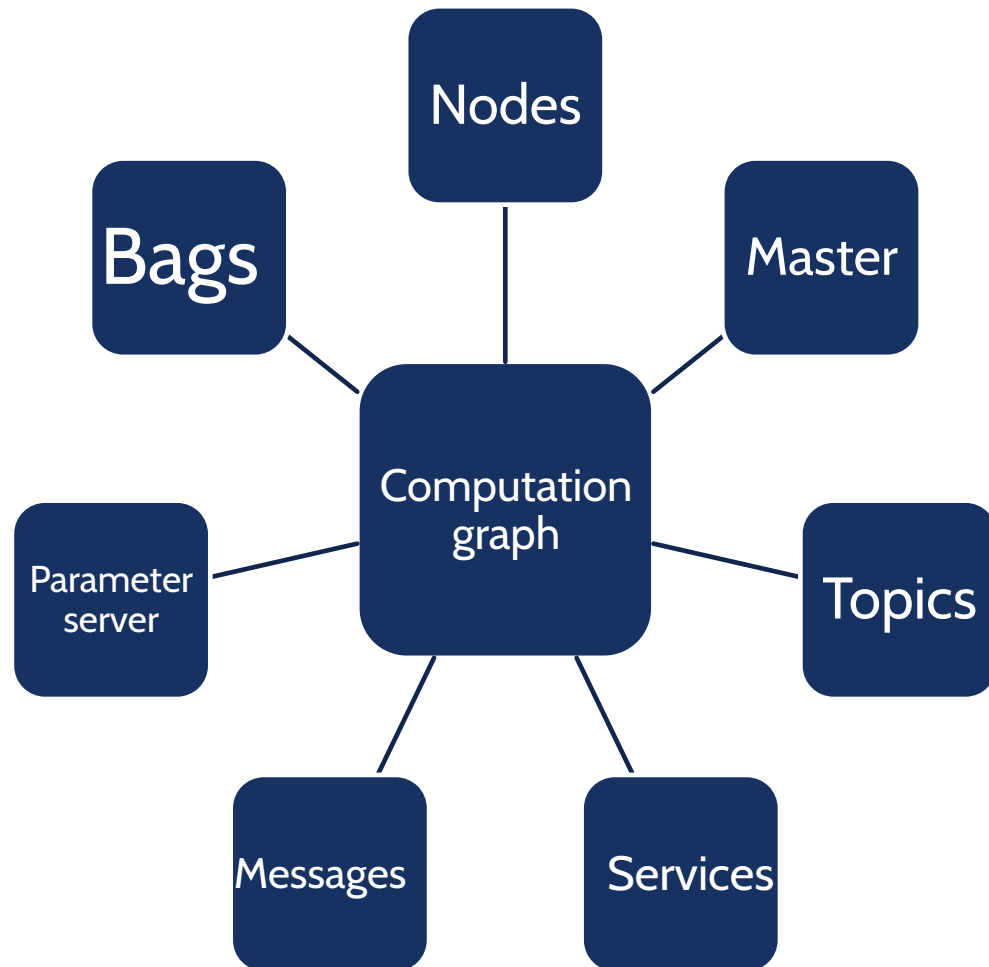
Packages

Monitoring and GUIs

Data Logging



ROS STRUCTURE: COMPUTATIONAL GRAPH



The *Computation Graph* is the peer-to-peer network of ROS processes that are processing data together.

NODES



Executable unit of ROS:

- Scripts for Python

- Compiled source code for C++

Process that performs computation

Nodes exchange information via the graph

Meant to operate at fine-grained scale

A robot system is composed by various nodes

```
roslaunch package_name node_name
```

```
roslaunch turtlesim turtlesim_node
```

MASTER



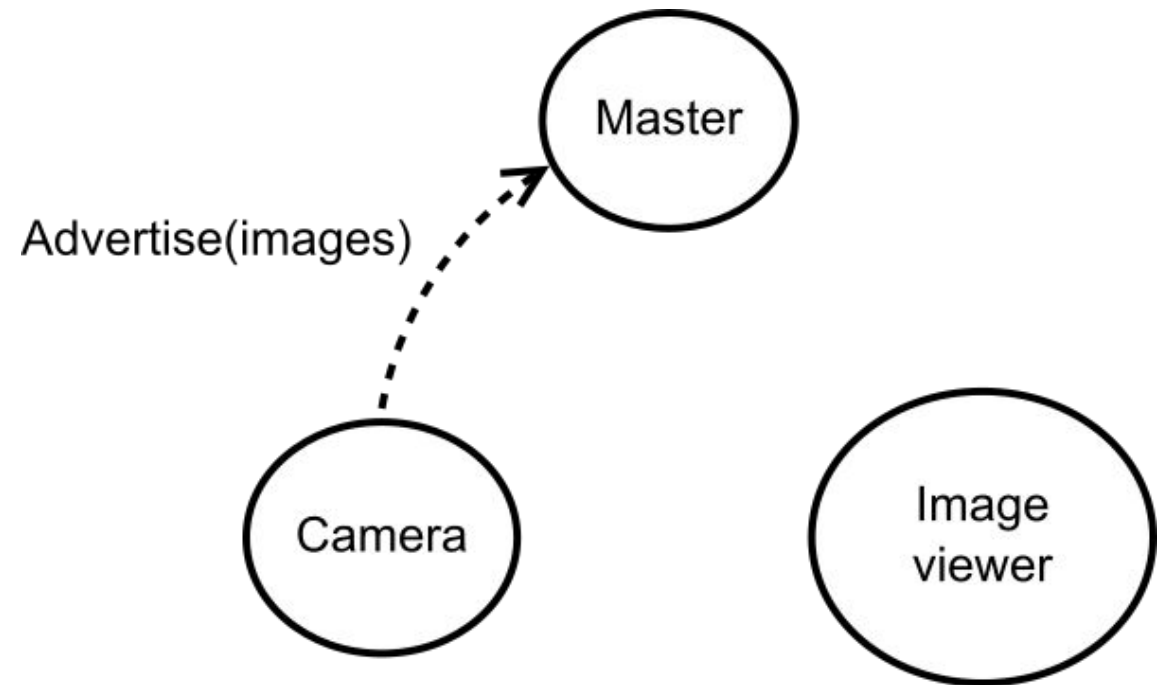
Provides naming and registration services

Essential for nodes interactions

One master for each system, even on distributed architectures

Enables individual ROS nodes to locate one another

One of the functionalities provided by roscore



MASTER



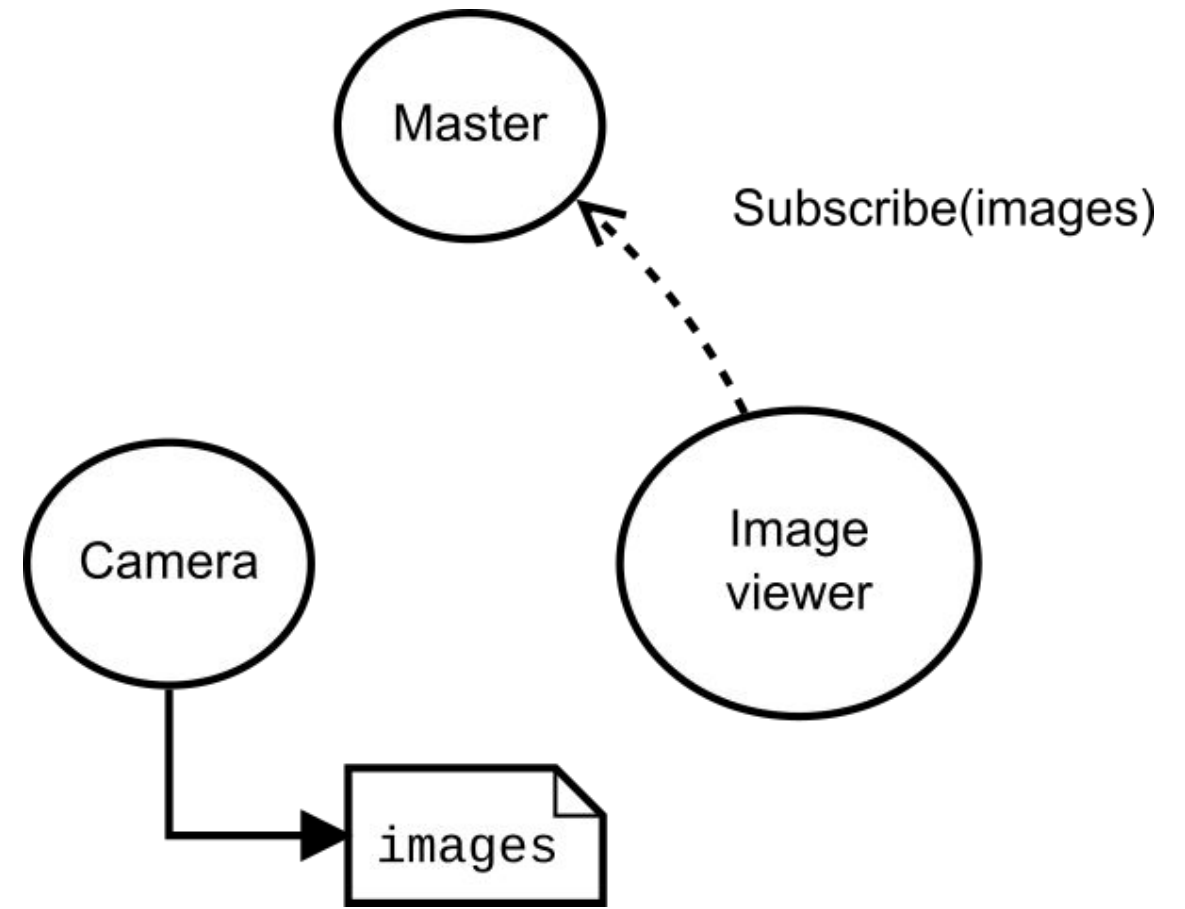
Provides naming and registration services

Essential for nodes interactions

One master for each system, even on distributed architectures

Enables individual ROS nodes to locate one another

One of the functionalities provided by roscore



MASTER



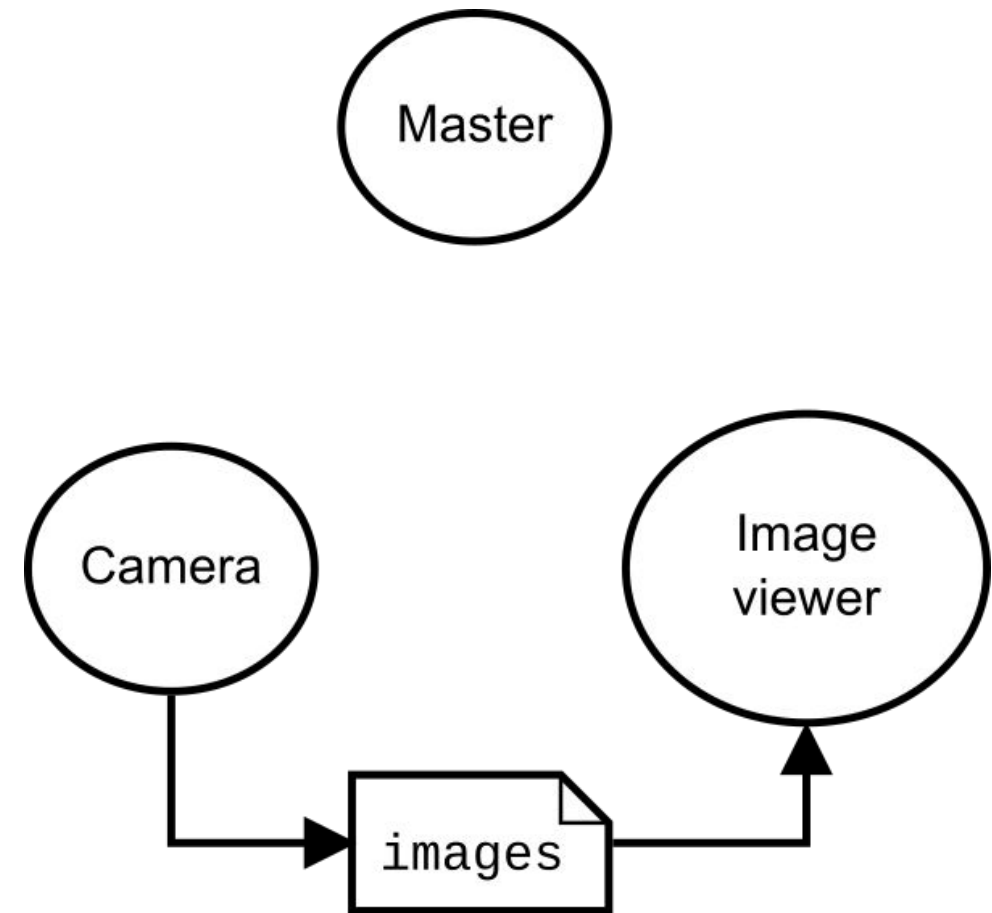
Provides naming and registration services

Essential for nodes interactions

One master for each system, even on distributed architectures

Enables individual ROS nodes to locate one another

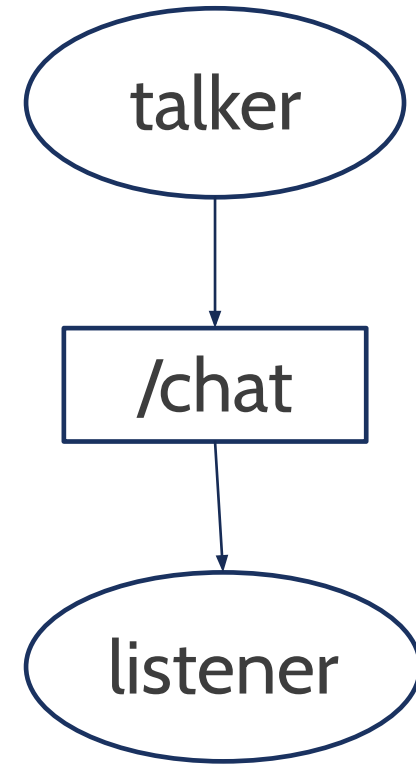
One of the functionalities provided by roscore



TOPICS



- Named channels for communication
- Implement the publish/subscribe paradigm
- No guarantee of delivery
- Have a specific message type
- Multiple nodes can publish messages on a topic
- Multiple nodes can read messages from a topic



TOPICS



Named channels for communication

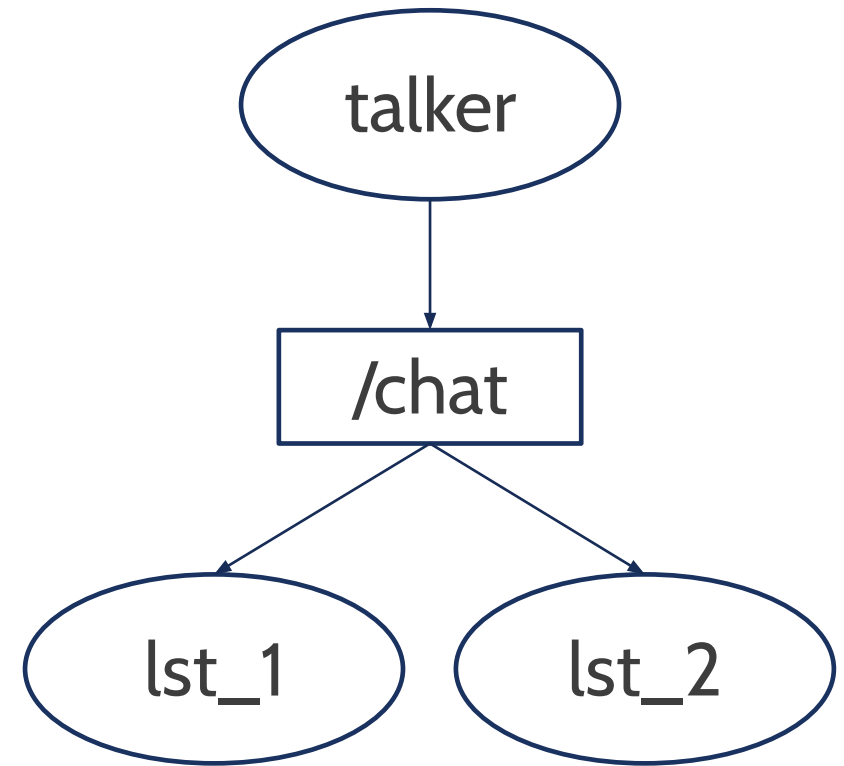
Implement the publish/subscribe paradigm

No guarantee of delivery

Have a specific message type

Multiple nodes can publish messages on a topic

Multiple nodes can read messages from a topic



TOPICS



Named channels for communication

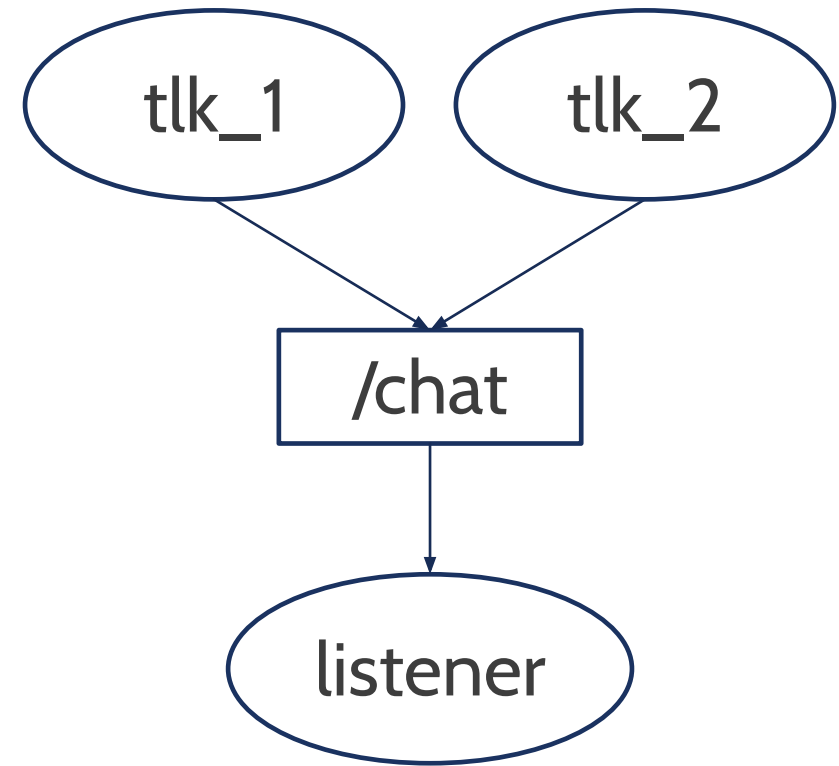
Implement the publish/subscribe paradigm

No guarantee of delivery

Have a specific message type

Multiple nodes can publish messages on a topic

Multiple nodes can read messages from a topic



MESSAGES



Messages are exchanged on topics

They define the type of the topic

Various already available messages

It is possible to define new messages using a simple language

Existing message types can be used in new messages together with base types

`std_msgs/Header.msg`

`uint32 seq`
`time stamp`
`string frame_id`

`std_msgs/String.msg`

`string data`

`sensor_msgs/Joy.msg`

`std_msgs/Header header`
`float32[] axes`
`int32[] buttons`

MESSAGES



Messages are exchanged on topics

They define the type of the topic

Various already available messages

It is possible to define new messages using a simple language

Existing message types can be used in new messages together with base types

Quick recap:

14 base types

32 std_msgs

29 geometry_msgs

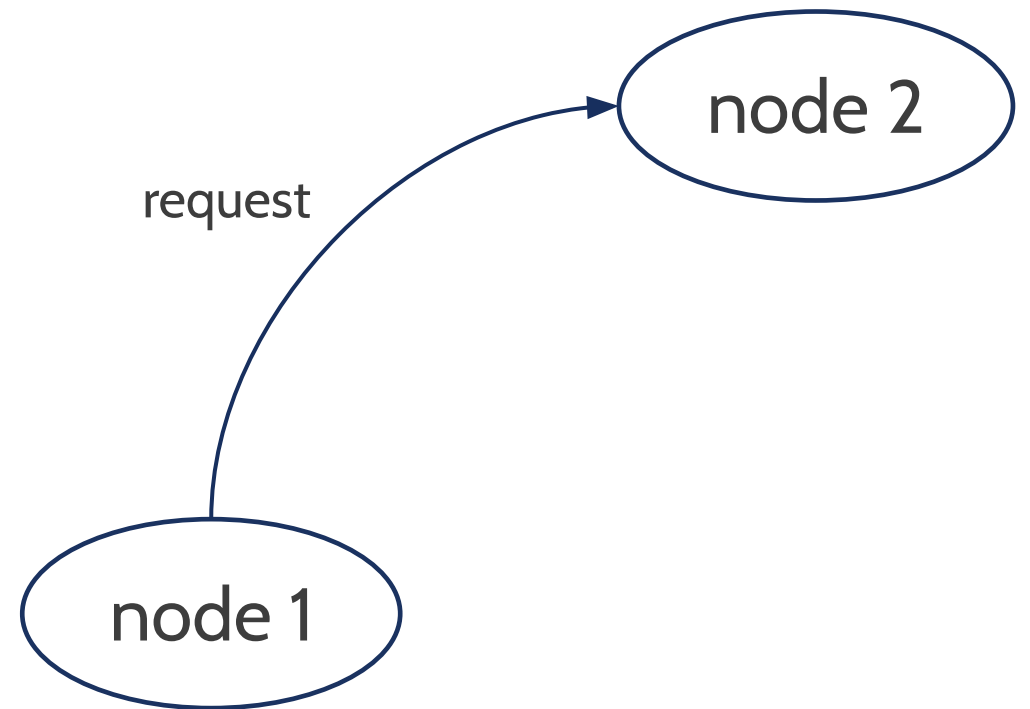
26 sensor_msgs

...and more

SERVICES



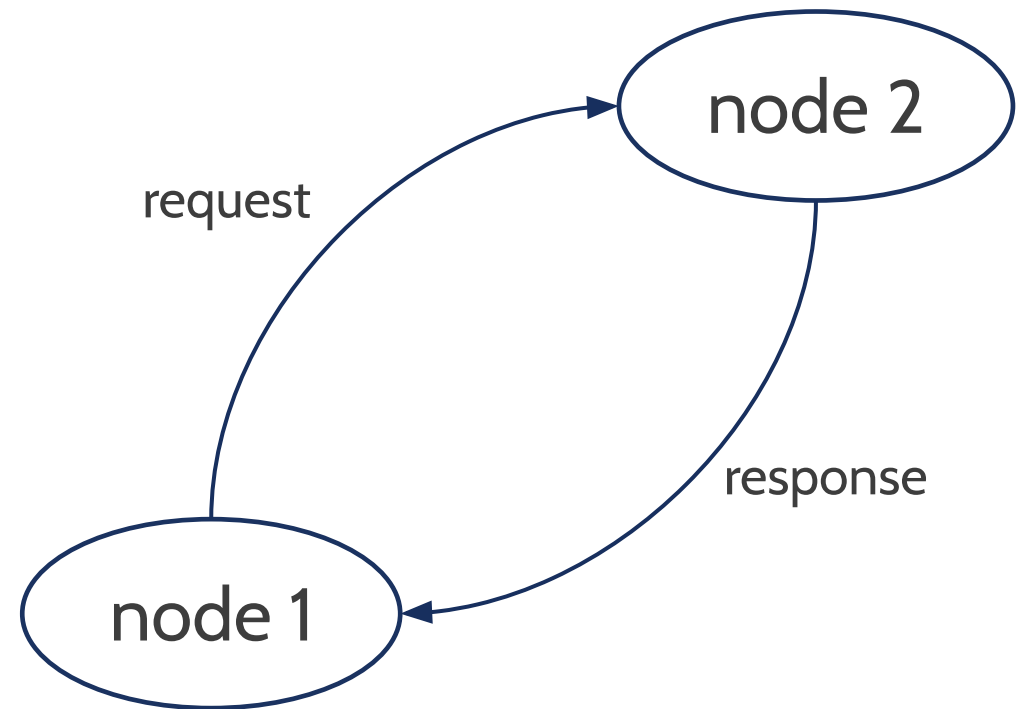
Work like remote function calls
Implement the client/server paradigm
Code waits for service call to complete
Guarantee of execution



SERVICES



Work like remote function calls
Implement the client/server paradigm
Code waits for service call to complete
Guarantee of execution



SERVICES



Work like remote function calls

Implement the client/server paradigm

Code waits for service call to complete

Guarantee of execution

Use of message structures

example/AddTwoInt.srv

int64 A

int64 B

int64 Sum

} request

} response

PARAMETER SERVER



Shared, multivariable dictionary that is accessible via network

Nodes use this server to store and retrieve parameters at runtime

Not designed for performance, not for data exchange

Connected to the master, one of the functionalities provided by roscore

name	value
/gains/P	10.0
/gains/I	1.0
/gains/D	0.1
use_sim_time	True

```
rosparm [set|get] name value
```

```
rosparm set use_sim_time True
```

```
rosparm get use_sim_time
```

```
> True
```

PARAMETER SERVER



Shared, multivariable dictionary that is accessible via network

Nodes use this server to store and retrieve parameters at runtime

Not designed for performance, not for data exchange

Connected to the master, one of the functionalities provided by roscore

Available types:

32-bit integers

Booleans

Strings

Doubles

ISO8601 dates

Lists

Base64-encoded binary data



File format (*.bag) for storing and playing back messages

Primary mechanism for data logging

Can record anything exchanged on the ROS graph (messages, services, parameters, actions)

Important tool for analyzing, storing, visualizing data and testing algorithms.

```
rosv bag record -a
```

```
rosv bag record /topic1 /topic2
```

```
rosv bag play ~/bags/fancy_log.bag
```

```
rqt_bag ~/bags/fancy_log.bag
```



roscore is a collection of nodes and programs that are pre-requisites of a ROS-based system

Must be running in order for ROS nodes to communicate

Launched using the roscore command.

Elements of roscore:

- a ROS Master

- a ROS Parameter Server

- a roscore logging node

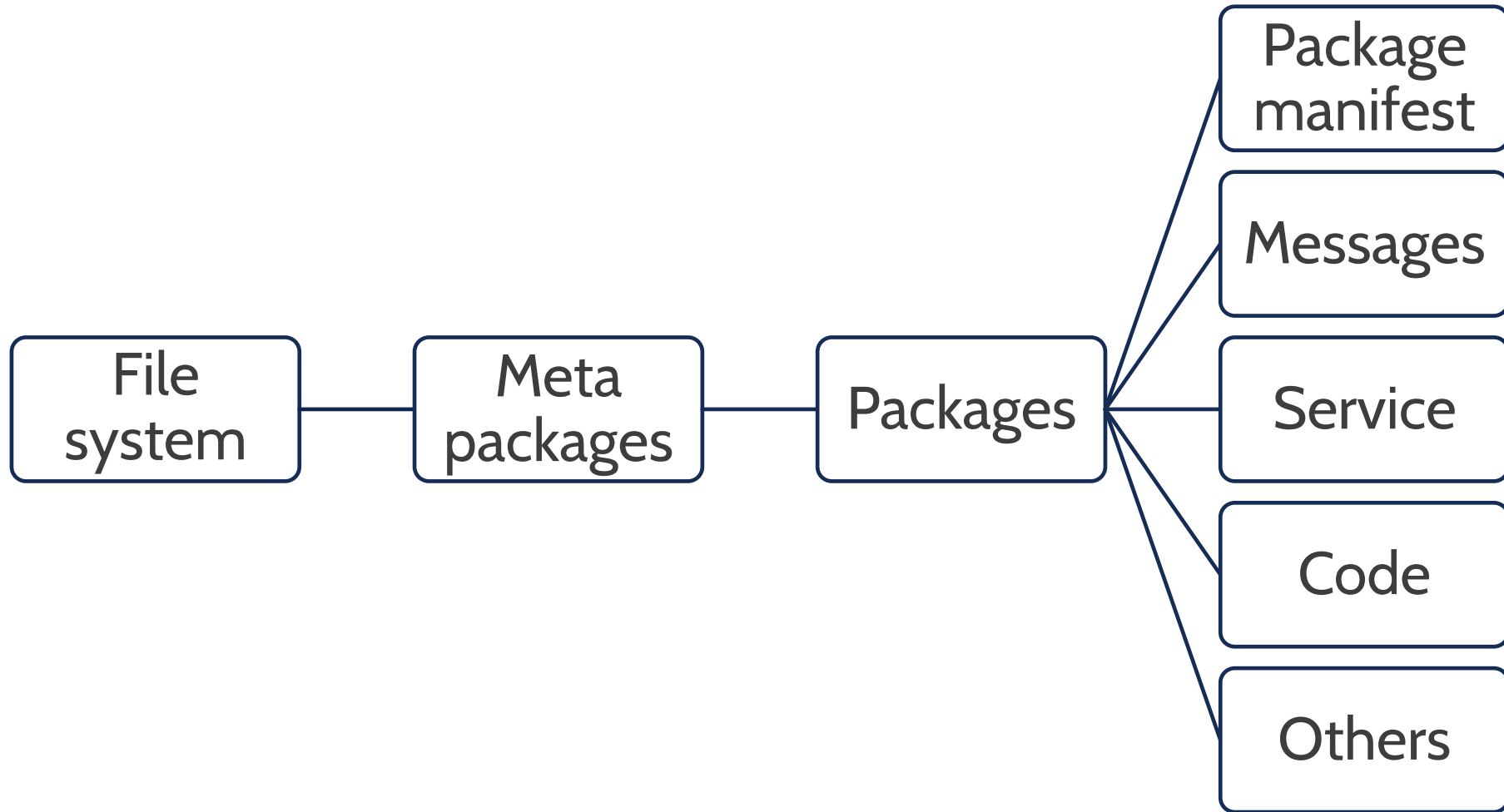
ROS FILESYSTEM

ROBOTICS

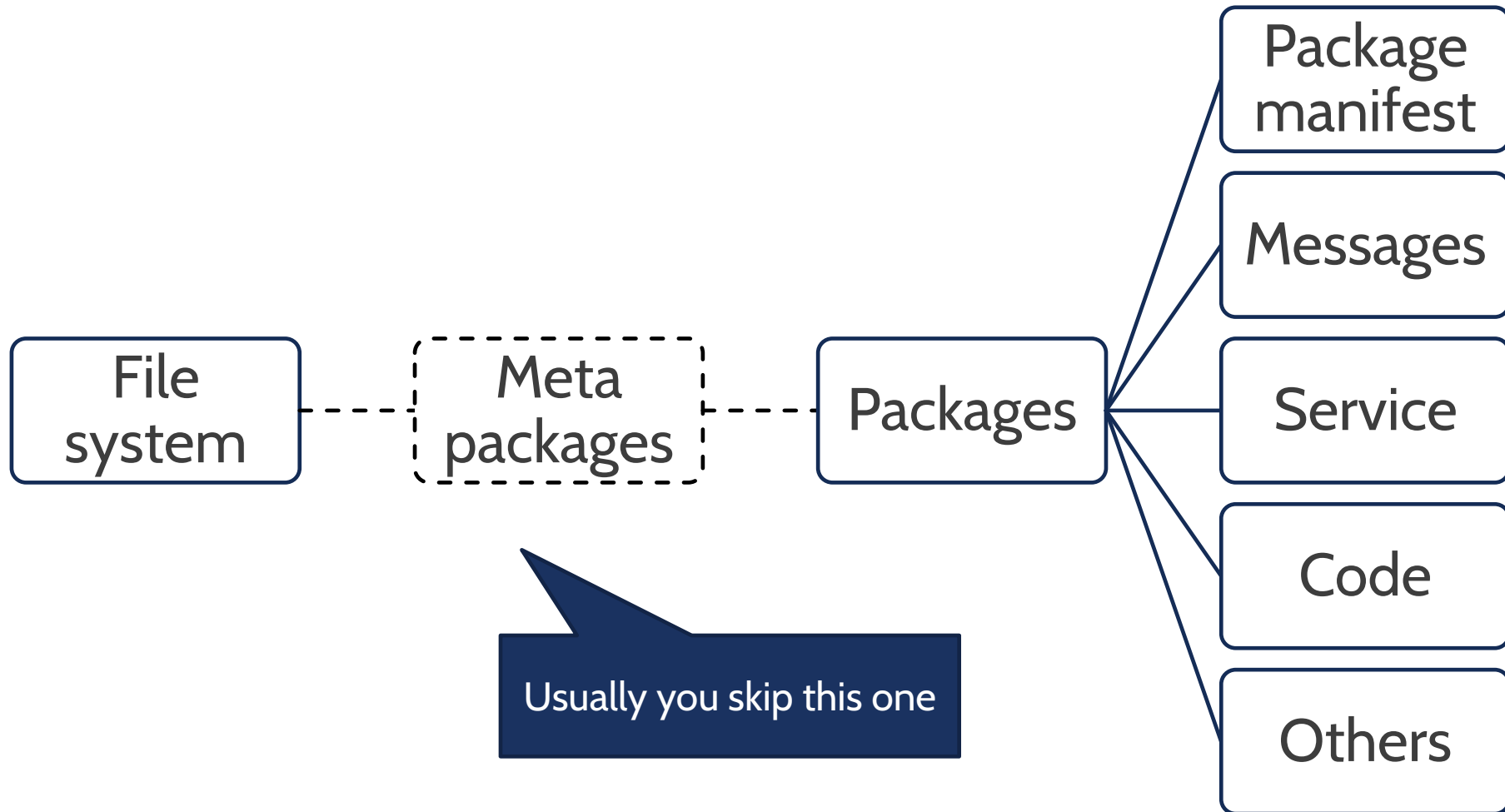


POLITECNICO
MILANO 1863

ROS FILE SYSTEM



ROS FILE SYSTEM



PACKAGES AND METAPACKAGES



PACKAGES

- Atomic element of ROS file system
- Used as a reference for most ROS commands
- Contains nodes, messages and services
- package.xml used to describe the package
- Mandatory container

METAPACKAGES

- Aggregation of logical related elements
- Not used when navigating the ROS file system
- Contains other packages
- package.xml used to describe the package
- Not required



STRUCTURE OF A PACKAGE

Folder structure:

/src, /include, /scripts (coding)

/launch (launch files)

/config (configuration files)

Required files:

CMakeList.txt: Build rules for catkin

package.xml: Metadata for ROS

